

Operating System

Assignment - 3

Short Answer type - Part A

1. Example - Two people try to withdraw money from the same bank ATM account simultaneously; both read the same balance and both withdraw, causing overdraft.
Mutual exclusion: Lock the account while one transaction runs (only one person can access / update balance at a time). This prevents interleaved reads/writes and preserves correctness.
2. • Peterson's: Simple to implement in software for two processes; relies on atomic reads/writes and memory ordering (so hardware memory model matters).
• Semaphores: higher-level, support many processes, implemented using OS primitives (atomic test and set or hardware support) - more complex but portable and scalable.
Peterson's is simple but hardware-sensitive & limited; semaphores are robust and general-purpose.
3. Monitors integrate mutual exclusion with condition variables and let the runtime handle blocking/wakeup efficiently (avoids busy-waiting), reducing cache-coherence traffic and context-switch overhead on multi-core CPUs.

4. Starvation : continuous arrival of readers can starve writers (writers never get exclusive access).

Prevention : use writer-priority or fair-reader-writer locks (e.g. queue requests or give waiting writers precedence) so writers eventually acquire the lock.

5. If processes must request all needed resources at start (to eliminate hold and wait), resources may be underutilized and processes may block or fail to start (reduced concurrency and throughput); also increases likelihood of resource starvation and wasted allocation.

Part B : Application / Numerical Based

6. (a) Need = Max - Allocation

- $P_0 : (7-0, 5-1, 3-0) = (7, 4, 3)$
- $P_1 : (3-2, 2-0, 2-0) = (1, 2, 2)$
- $P_2 : (9-3, 0-0, 2-2) = (6, 0, 0)$
- $P_3 : (4-2, 2-1, 2-1) = (2, 1, 1)$
- $P_4 : (5-0, 3-0, 3-2) = (5, 3, 1)$

- (b) Total Allocations = A: 7, B: 2, C: 5 $\rightarrow$ Available = $(10-7, 5-2, 7-5) = (3, 3, 2)$

Find a safe sequence: $P_1 \rightarrow P_3 \rightarrow P_0 \rightarrow P_2 \rightarrow P_4$
(each's Need $\leq$ current Available as you step).
So the system is safe.

Request $\leq$ Need ?

(c) Yes $(1, 0, 2) \leq (1, 2, 2)$

Request $\leq$ Available ? Yes $(1, 0, 2) \leq (3, 3, 2)$

Tentatively allocate $\rightarrow$ new Available $= (2, 3, 0)$. A safe sequence exists (P_1 finishes $\rightarrow$ releases $\rightarrow$ then P_3, P_0, P_2, P_4)
Request can be granted immediately.

7. Deadlock situation (5 philosophers)

- Each philosopher picks left fork first.
- All 5 hold one fork and wait for the right fork $\rightarrow$ circular wait.
- No one can proceed $\rightarrow$ deadlock.

Method to prevent deadlock:

- Use a room semaphore = 4 (allow ≤ 4 philosophers to try picking forks).
- Or alternatively: make one philosopher pick right fork first (break symmetry).

8. Given: device = 500 KB/s, block = 100 bytes, interrupt time = 5 μ s.

Assume 1 KB = 1024 bytes $\rightarrow$ transfer = $500 \times 1024 = 512000$ B/s.

Interrupts/sec = $512000 / 100 = 5120$ interrupts/s.

CPU time spent = $5120 \times 5 \mu\text{s} = 25,600 \mu\text{s} = 25.6 \text{ ms/s} =$

2.56% CPU.

Improvement: use DMA (or larger block transfers / interrupt coalescing) so the CPU is interrupted far less often while keeping the same throughput.

9. (a) Critical sections & IPC

- Critical sections: shared radar buffer/queue (raw sensor frames), shared flight-plan database, and message channel to pilot comms (to avoid interleaved writes).
- IPC choice: real-time shared memory + priority-aware mutexes or lock-free queues (for lowest latency) or POSIX real-time message queues if message ordering and priorities are needed.

(b) Deadlock detection & recovery (minimal disruption)

- Detection: maintain a real-time wait for graph or resource allocation table and run lightweight periodic detection (or use timeouts).
- Recovery: preempt or roll back the least-critical or lowest-priority process (checkpoint ~~state~~), release its resources, and resume higher-priority tasks; notify operators and reassign tasks. Use checkpointing / transaction logs to minimize lost work.